

Threat Modelling and Risk Assessment

IE3142 DevOps Security — Web Goat Security Pipeline

Scope

The threat model focuses on the security of the pipeline developed around the OWASP WebGoat application. The analysis encompasses the application's containerized runtime environment, authentication and data access mechanisms, and the local Docker based deployment represented in the project architecture. The vulnerabilities intentionally incorporated into Web Goat as security training exercises are considered the primary attack surface for this assessment.

STRIDE Threat Identification

4 realistic and application specific threats were identified within the WebGoat environment. Each threat corresponds to a vulnerability class demonstrated during the exploit-and-fix activities. The identified threats were analyzed using the STRIDE threat modelling framework and were assessed according to their likelihood of exploitation and potential impact.

Table 1: STRIDE Based Threat Identification for the WebGoat Pipeline

Threat	Description (WebGoat-specific)	STRIDE category	Likelihood	Impact	Justification
T1 — SQL Injection	The WebGoat SQL Injection lesson constructs database queries by concatenating unsensitized user input. Consequently, an attacker may manipulate the query logic and potentially retrieve unauthorized information from the database.	Tampering / Information Disclosure	High	High	No special skill is required to exploit unsensitized string-built queries, and a successful attack can expose the entire underlying database.
T2 — Stored Cross-Site Scripting (XSS)	User-supplied content, such as comments or profile information, is stored and subsequently rendered to other users without appropriate output encoding. This allows a malicious script to execute within another user's browser session.	Tampering / Information Disclosure	High	Medium–High	The vulnerable field accepts and stores arbitrary input with no encoding, and any user viewing the page triggers the payload; impact is bounded to session/cookie theft rather than full system compromise.

T3 — JWT Bypass	The WebGoat JWT lesson demonstrates insufficient or absent token-signature verification, including the alg:none attack scenario. This weakness may allow an attacker to forge a JSON Web Token and impersonate another user.	Spoofing / Elevation of Privilege	Medium	High	Exploitation requires understanding token structure, which is a moderate barrier, but a successful forgery grants full impersonation, including of privileged accounts.
T4 — Insecure Direct Object Reference (IDOR)	The WebGoat Broken Access Control lesson does not adequately verify object ownership on the server side. As a result, a user may access another user's data by modifying an object identifier within a request.	Elevation of Privilege / Information Disclosure	Medium	Medium-High	The attack requires only parameter manipulation with no special tooling, though impact is limited to the data reachable through the affected endpoint rather than the whole system.

Risk Assessment

The identified threats were assessed using a three-level scale consisting of Low, Medium, and High for both likelihood and impact. The resulting ratings were plotted using a risk matrix. The assessments reflect the relative ease with which each vulnerability can be exploited within the deployed WebGoat environment and the potential severity of the resulting security compromise.

Table 2 : Risk Matrix

Likelihood \ Impact	Low	Medium	High
High	—	—	T1 SQLi T2 XSS
Medium	—	T4 IDOR	T3 JWT bypass
Low	—	—	—

The risk matrix indicates that SQL Injection and Stored XSS have a high likelihood of exploitation and therefore represent significant combined risk within the assessed environment. In comparison, the JWT bypass has a high potential impact despite having a moderate exploitation barrier. The IDOR vulnerability presents a medium likelihood and medium-to-high potential impact.

Threat-to-Control Mapping

Each identified threat is associated with a corresponding security control implemented either through secure-coding remediation or through automated security gates within the CI/CD pipeline. The following mapping identifies the principal control applied to each vulnerability and its implementation location.

Table 3: Threat-to-Control Mapping

Threat	Control	Category	Where it lives
T1 — SQL Injection	Parameterized queries (PreparedStatement)	Secure coding + SAST	The vulnerability is remediated within the Java source code of the SQL Injection lesson and subsequently verified using Semgrep through the SAST security gate in the CI pipeline.
T2 — Stored XSS	Output encoding of user-supplied content	Secure coding + SAST	The vulnerability is remediated within the Java source code of the XSS lesson and verified using Semgrep through the SAST security gate in the CI pipeline.
T3 — JWT Bypass	Enforced signature verification; reject alg:none; validate issuer/expiry	Secure coding + dependency scanning	The vulnerability is addressed within the authentication logic of the JWT lesson. Supporting library versions are additionally assessed through the dependency-scanning security gate
T4 — IDOR	Server-side ownership check on every object request	Secure coding	The vulnerability is addressed within the request-handling logic of the Broken Access Control lesson by enforcing appropriate server-side ownership checks.

Summary

The threat model identified four primary vulnerabilities within the WebGoat environment: SQL Injection, Stored XSS, JWT Bypass, and Insecure Direct Object Reference (IDOR). These threats cover multiple STRIDE categories, including Tampering, Information Disclosure, Spoofing, and Elevation of Privilege, thereby providing coverage across different security threat types.

The identified vulnerabilities also correspond to three OWASP Top 10 (2021) categories: Injection, Broken Authentication, and Broken Access Control. This distribution provides broader security coverage rather than focusing exclusively on a single vulnerability class.